

Mid-Level DevOps, Cloud & Platform Engineering

Interview Questions & Answers Guide

94 curated Q&A across **11 topics**: Kubernetes, Helm, Networking, CI/CD Fundamentals, GitLab CI/CD, Jenkins, Secrets Management, Terraform, Terragrunt, OpenTofu, ArgoCD & GitOps

Each answer is written at the depth a 3-6 year engineer is expected to demonstrate: clear definitions, the reasoning behind the design, and the kind of production/troubleshooting detail that signals real hands-on experience rather than memorized definitions. Code snippets illustrate the most commonly referenced configuration patterns.

Generated June 2026

Table of Contents

01	Kubernetes (K8s)	11 Q&A
02	Helm Charts	10 Q&A
03	Networking	10 Q&A
04	CI/CD Fundamentals	8 Q&A
05	GitLab CI/CD Pipelines	8 Q&A
06	Jenkins	9 Q&A
07	Secrets Management	8 Q&A
08	Terraform	10 Q&A
09	Terragrunt	6 Q&A
10	OpenTofu	5 Q&A
11	ArgoCD & GitOps	9 Q&A

How to use this guide: read each answer once before your interview. The goal isn't to recite it verbatim, but to internalize the structure (definition, mechanism, real-world nuance) so you can reconstruct an equally solid answer under pressure, even if the exact question is phrased differently.

Kubernetes (K8s)

11 questions

Core concepts a mid-level engineer is expected to explain clearly and back with hands-on troubleshooting experience.

Q1. What is Kubernetes, and what problem does it solve?

Kubernetes is a container orchestration platform that automates deployment, scaling, networking, and lifecycle management of containerized applications across a cluster of machines. Instead of manually placing containers on servers, you describe the desired state (how many replicas, what image, what resources) in a manifest, and the control plane continuously reconciles the actual cluster state to match it. It solves problems like self-healing (restarting failed containers), horizontal scaling, service discovery, load balancing, rolling updates/rollbacks, and efficient bin-packing of workloads onto nodes.

Q2. Explain the relationship between Pods, ReplicaSets, and Deployments.

A Pod is the smallest deployable unit — one or more tightly coupled containers sharing network and storage. A ReplicaSet ensures a specified number of identical Pod replicas are running at all times, replacing any that fail. A Deployment sits on top of a ReplicaSet and manages it declaratively, giving you rolling updates, rollback history, and pause/resume of updates. In practice you almost never create a Pod or ReplicaSet directly — you create a Deployment, and Kubernetes manages the ReplicaSet and Pods underneath it.

deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
        - name: web
          image: myrepo/web-app:1.4.0
          resources:
            requests: {cpu: "100m", memory: "128Mi"}
            limits: {cpu: "500m", memory: "256Mi"}
```

Q3. What are the main Service types and when do you use each?

ClusterIP (default) exposes the service only inside the cluster on a virtual IP — used for internal service-to-service traffic. NodePort opens a static port on every node and forwards to the service, mainly for quick testing or on-prem setups without a cloud load balancer. LoadBalancer provisions an external cloud

load balancer (ELB/ALB, Azure LB, GCP LB) and is the standard way to expose a service to the internet on cloud platforms. ExternalName maps a service to an external DNS name with no proxying, useful for referencing external systems by an in-cluster name.

Q4. Deployment vs StatefulSet vs DaemonSet — what's the difference?

A Deployment is for stateless, interchangeable Pods where identity doesn't matter — any replica can be killed and replaced freely. A StatefulSet is for stateful workloads (databases, Kafka, Zookeeper) that need stable network identity (predictable Pod names like db-0, db-1), stable persistent storage per Pod, and ordered, graceful scaling/rolling updates. A DaemonSet ensures exactly one Pod runs on every (or a selected subset of) node — typically used for node-level agents like log collectors, monitoring exporters, or CNI/CSI components.

Q5. What's the difference between a ConfigMap and a Secret, and how do they get into a Pod?

Both decouple configuration from container images. A ConfigMap holds non-sensitive key/value configuration (feature flags, URLs, config files); a Secret holds sensitive data (passwords, tokens, certs) and is stored base64-encoded — note this is encoding, not encryption, so Secrets need RBAC, encryption-at-rest, and ideally an external secret manager for real protection. Both can be mounted into a Pod as environment variables or as files via a volume mount, and updates to a mounted ConfigMap/Secret can propagate to the Pod without a restart (env vars require a restart to pick up changes).

pod-snippet.yaml

```
envFrom:
- configMapRef:
  name: app-config
volumes:
- name: secret-vol
  secret:
    secretName: db-credentials
containers:
- name: app
  volumeMounts:
  - name: secret-vol
    mountPath: /etc/secrets
    readOnly: true
```

Q6. Explain liveness, readiness, and startup probes.

A liveness probe tells Kubernetes whether a container is still healthy — if it fails, the kubelet kills and restarts the container. A readiness probe tells Kubernetes whether the Pod is ready to receive traffic — if it fails, the Pod is removed from the Service's endpoint list but not restarted, which prevents sending requests to a Pod that's still warming up or temporarily overloaded. A startup probe gives slow-starting containers extra time before liveness/readiness checks kick in, preventing premature restarts during a long boot sequence (e.g., JVM apps with heavy initialization).

Q7. How does Kubernetes scheduling decide where a Pod runs? (requests/limits, affinity, taints/tolerations)

The scheduler filters nodes that have enough allocatable CPU/memory to satisfy a Pod's resource requests, then scores remaining nodes (spread, affinity rules, etc.) to pick the best fit. Resource requests guarantee a minimum; limits cap usage (and can trigger OOMKill or CPU throttling if exceeded). Node/Pod affinity and anti-affinity let you attract or repel Pods relative to nodes or other Pods (e.g., spread replicas across availability zones). Taints on a node repel Pods unless they carry a matching toleration — commonly used to reserve nodes for specific workloads (e.g., GPU nodes, dedicated namespaces).

Q8. Ingress vs Service — what role does each play?

A Service load-balances traffic across a set of Pods, mostly at L4 (TCP/UDP), and gives them a stable internal endpoint. An Ingress is an L7 routing rule set (host/path-based routing, TLS termination) that sits in front of one or more Services, and it requires an Ingress Controller (NGINX, Traefik, ALB Ingress Controller, etc.) actually running in the cluster to implement those rules — the Ingress resource itself is just configuration, not a running proxy.

ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web-ingress
  annotations:
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
spec:
  rules:
  - host: app.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: web-app
            port:
              number: 80
```

Q9. A Pod is stuck in CrashLoopBackOff. Walk through how you'd troubleshoot it.

First check `kubectl describe pod` for events (image pull errors, OOMKilled, failed probes) and `kubectl logs <pod> --previous` to see why the last container instance exited. Common causes are: the app crashing on a bad config/missing env var/Secret, a failing liveness probe restarting a healthy-but-slow app, insufficient memory causing OOMKill, or a bad image/entrypoint. If logs are empty, the container may be crashing before logging starts — exec into a sibling pod or temporarily override the command to debug interactively. Also check resource limits, dependent services (DB not reachable), and whether the issue is environment-specific (works in staging, fails in prod due to network policy or secret differences).

Q10. What is a Horizontal Pod Autoscaler (HPA) and how does it work?

HPA automatically adjusts the number of Pod replicas in a Deployment/StatefulSet based on observed metrics — by default CPU/memory utilization via the metrics-server, but it can also scale on custom or external metrics (queue length, requests-per-second) via the metrics API/adapters like Prometheus Adapter or KEDA. It periodically compares current metric values against a target, calculates the desired replica

count, and updates the Deployment's replica count within configured min/max bounds — useful for handling variable load without manual intervention or over-provisioning.

Q11. How do rolling updates and rollbacks work in a Deployment?

When you update a Deployment's Pod template (e.g., new image tag), Kubernetes creates a new ReplicaSet and gradually scales it up while scaling the old one down, controlled by `maxSurge` (extra Pods allowed above desired count) and `maxUnavailable` (how many can be down at once) — this keeps the app available throughout the rollout. Each revision is tracked in rollout history, so `kubectl rollout undo deployment/x` reverts to the previous ReplicaSet's Pod template instantly without needing to redeploy old artifacts manually. `kubectl rollout status` and `kubectl rollout history` are the go-to commands for monitoring and auditing this.

Helm Charts

10 questions

Helm is the de-facto package manager for Kubernetes; expect questions on chart structure, templating, lifecycle, and multi-environment patterns.

Q1. What problem does Helm solve, and what is a Chart?

Raw Kubernetes YAML doesn't scale well across environments — you end up duplicating manifests with small differences (replica counts, image tags, resource limits) per environment. Helm packages a set of related Kubernetes manifests into a versioned, parameterized unit called a Chart, using Go templating so the same chart can be rendered differently per environment by swapping a small values file. Helm also tracks releases (an installed instance of a chart), so you get install/upgrade/rollback as first-class operations instead of manually diffing and reapplying YAML.

Q2. Describe the standard structure of a Helm chart.

A chart directory contains Chart.yaml (chart metadata: name, version, appVersion, dependencies), values.yaml (default configuration values), a templates/ directory holding the Kubernetes manifest templates (Deployment, Service, Ingress, etc.) written with Go templating and Helm's built-in objects (.Values, .Release, .Chart), an optional charts/ directory for vendored subcharts/dependencies, and optionally a templates/NOTES.txt shown after install, plus a templates/_helpers.tpl for reusable named templates.

chart-layout

```
mychart/
├── Chart.yaml
├── values.yaml
├── charts/           # subcharts/dependencies
├── templates/
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── ingress.yaml
│   ├── _helpers.tpl
│   └── NOTES.txt
```

Q3. What are Helm hooks, and where would you use one?

Hooks let you run Kubernetes resources (typically Jobs) at specific points in a release's lifecycle — e.g., pre-install, post-install, pre-upgrade, post-upgrade, pre-delete. They're declared via an annotation on a manifest rather than a separate mechanism. A common use case is running a database migration Job before the new application version's Pods start (pre-upgrade), or sending a notification after a successful deploy (post-install).

hook-annotation.yaml

```
metadata:
  annotations:
    "helm.sh/hook": pre-upgrade
    "helm.sh/hook-weight": "0"
    "helm.sh/hook-delete-policy": before-hook-creation
```

Q4. Difference between helm install, upgrade, and rollback.

`helm install` creates a brand-new release from a chart. `helm upgrade` applies changes to an existing release — new values, a new chart version, or both — and Helm computes a diff against the currently deployed manifests to decide what to patch, replace, or leave alone (often combined with `--install` to make it idempotent: install if missing, upgrade if present). `helm rollback <release> <revision>` reverts a release to a previously recorded revision, which is possible because Helm stores release history (as Secrets/ConfigMaps by default) for every revision.

Q5. How do you manage the same chart across dev/staging/prod with different configuration?

Keep one chart and layer environment-specific values files on top of values.yaml, then pass them in order of precedence at install/upgrade time — later files override earlier ones, and `--set` overrides everything. This avoids duplicating templates per environment while still letting each environment tune replica counts, resource limits, ingress hosts, and feature flags independently.

multi-env.sh

```
helm upgrade --install web ./mychart \
  -f values.yaml \
  -f values-prod.yaml \
  --set image.tag=1.4.0 \
  --namespace prod
```

Q6. What is a chart dependency (subchart), and how do you manage them?

A chart can depend on other charts (e.g., a Redis or PostgreSQL chart) declared in the `dependencies` section of Chart.yaml with a name, version, and repository. Running `helm dependency update` downloads those charts into the local `charts/` directory so the parent chart is self-contained at install time. Values for a subchart are namespaced under its name in the parent's values.yaml, so you configure it like `redis:\n auth:\n enabled: true` instead of editing the subchart directly.

Q7. Your Helm release deploys but the app misbehaves. How do you debug the rendered manifests before they ever hit the cluster?

`helm template` renders the chart to plain YAML locally with no cluster interaction — good for a quick sanity check or diffing against a previous version. `helm install/upgrade --dry-run --debug` goes further: it talks to the cluster for things like validation/CRD lookups and prints the fully rendered manifests plus the computed values, which is the fastest way to catch a templating bug (wrong indentation, a missing value defaulting to empty, a typo'd field name) before committing to a real release.

debug.sh

```
helm template ./mychart -f values-prod.yaml | less
helm upgrade web ./mychart --dry-run --debug -f values-prod.yaml
```

Q8. How would you handle secrets that need to go into a Helm chart's values?

Never commit real secret values into values.yaml in plaintext. Common patterns: keep secrets out of the chart entirely and reference a Kubernetes Secret created separately (by External Secrets Operator pulling from Vault/AWS Secrets Manager); use `helm-secrets` (a plugin wrapping SOPS) to encrypt values files at rest in Git and decrypt them transparently at deploy time; or use Sealed Secrets, where the chart ships an encrypted SealedSecret manifest that only the in-cluster controller can decrypt. The right choice usually depends on whether you already run Vault/cloud secret managers in the org.

Q9. What is Helmfile and why might a team use it on top of Helm?

Helm itself manages one release at a time. Helmfile is a declarative spec (helmfile.yaml) for managing many Helm releases together — across charts, repos, and environments — with templating, environment selection, and dependency ordering between releases. It's especially useful when a platform consists of dozens of charts (ingress controller, cert-manager, monitoring stack, and N application charts) that need to be deployed consistently as one operation, similar to how Terragrunt sits on top of Terraform.

Q10. How does Helm chart versioning work, and why does it matter for CI/CD?

Chart.yaml has two separate version fields: `version` (the chart's own SemVer, bumped whenever the templates/values change) and `appVersion` (the version of the application the chart deploys, purely informational/metadata). In CI/CD, pipelines typically bump the chart version on every merge, package it with `helm package`, push it to a chart repository (ChartMuseum, OCI registry, or a Git-based repo), and downstream environments pin to a specific chart version for reproducible, auditable deployments rather than always pulling 'latest'.

Networking

10 questions

Foundational networking plus Kubernetes-specific networking — a frequent area where candidates lose points by only knowing one side.

Q1. TCP vs UDP — when would a system design favor one over the other?

TCP is connection-oriented and guarantees ordered, reliable delivery via handshakes, acknowledgements, and retransmission — used where correctness matters more than raw speed (HTTP/HTTPS, databases, SSH). UDP is connectionless with no delivery guarantees or ordering, trading reliability for lower latency and overhead — used for DNS queries, video/voice streaming, and protocols that implement their own reliability layer (like QUIC/HTTP3, which runs over UDP but adds its own guarantees).

Q2. Walk through what happens, network-wise, when a Kubernetes Service receives a request and routes it to a Pod.

A client hits the Service's ClusterIP (a virtual IP with no real interface behind it). kube-proxy, running on every node, watches the API server for Service/Endpoint changes and programs the node's iptables (or IPVS, depending on mode) rules so that traffic to that virtual IP gets DNAT'd to one of the healthy backing Pod IPs, selected via the Endpoints/EndpointSlice object. This is why a Pod must pass its readiness probe to be included as a destination — failing the probe removes it from the EndpointSlice and traffic stops being routed to it, without anyone touching the Service object itself.

Q3. What is a CNI plugin, and name a few you've encountered?

The Container Network Interface (CNI) is the spec/plugin model Kubernetes uses to assign Pod IPs and wire up Pod networking — the kubelet calls the configured CNI plugin whenever a Pod is created or deleted. Common implementations are Calico (BGP-based routing, strong NetworkPolicy support), Flannel (simpler overlay network, historically weaker policy support), and Cilium (eBPF-based, very strong observability/policy and increasingly used to replace kube-proxy entirely). The choice affects performance, supported NetworkPolicy features, and observability tooling available in the cluster.

Q4. How does DNS resolution work inside a Kubernetes cluster?

CoreDNS runs as a cluster add-on and is configured (via each Pod's `/etc/resolv.conf`, injected by the kubelet) as the nameserver for in-cluster lookups. Every Service automatically gets a DNS name in the form `<service>.<namespace>.svc.cluster.local`, which CoreDNS resolves to the Service's ClusterIP; Pods can also get DNS names if a headless Service is used, which is common for StatefulSets needing to address individual Pod instances directly. External DNS lookups from inside Pods are typically forwarded upstream from CoreDNS to the node's/cluster's configured resolvers.

Q5. Difference between a Load Balancer and a Reverse Proxy — and where does an Ingress Controller fit?

A Load Balancer distributes traffic across multiple backend instances, often at L4, primarily for availability and scaling, without necessarily understanding application-layer content. A Reverse Proxy operates at L7,

sits in front of one or more backends, and can do content-aware routing, TLS termination, caching, compression, and request rewriting. An Ingress Controller (NGINX, Traefik, HAProxy, etc.) is effectively an L7 reverse proxy purpose-built for Kubernetes — it watches Ingress resources and reconfigures itself to route based on host/path rules, often sitting behind a cloud L4 Load Balancer that gets the traffic into the cluster in the first place.

Q6. Explain CIDR notation and why it matters when sizing a VPC or cluster network.

CIDR (Classless Inter-Domain Routing) notation, like 10.0.0.0/16, specifies a network address plus a prefix length indicating how many leading bits are fixed (the network portion) versus variable (the host portion) — a /16 gives roughly 65k addresses, a /24 gives 256. Sizing matters a lot in Kubernetes because every Pod typically gets its own IP from the cluster's Pod CIDR; under-sizing it (e.g., a /24 per node in some CNI setups) can silently cap how many Pods a node can ever run, long before CPU/memory becomes the bottleneck — this is a classic 'works in dev, breaks at scale' production incident.

Q7. What is NAT, and where do you typically encounter it in cloud networking?

Network Address Translation rewrites source/destination IPs as traffic crosses a boundary, most commonly to let many private IPs share one public IP. In cloud VPCs, a NAT Gateway sits in a public subnet so that instances in private subnets can reach the internet (e.g., to pull packages or call external APIs) without themselves being directly reachable from the internet — outbound-only, which is a key part of a defense-in-depth network design.

Q8. Give a high-level rundown of what happens in a TLS handshake.

The client sends a ClientHello with supported TLS versions/cipher suites; the server responds with its certificate and chosen cipher suite. The client validates the certificate chain against trusted CAs, then client and server perform a key exchange (commonly via Diffie-Hellman/ECDHE for forward secrecy) to derive a shared symmetric session key. From that point on, all traffic is encrypted with the fast symmetric cipher, while the slower asymmetric crypto was only used to establish trust and exchange the key — this is why TLS termination at a load balancer or Ingress is comparatively expensive and often a target for session resumption/optimization.

Q9. Two Pods in the same cluster can't reach each other on a specific port. What's your troubleshooting flow?

First confirm both Pods are Running and check their actual IPs/ports with `kubectl exec ... -- curl/nc`. Check for a NetworkPolicy that might be denying the traffic — NetworkPolicies are deny-by-default once any policy selects a Pod, which surprises a lot of people. Verify the target container is actually listening on that port (`netstat`/`ss` inside the container) and that the Service's selector labels actually match the Pod's labels — a label typo silently produces an empty Endpoints list. Finally, check CNI-level issues (node-to-node connectivity, security groups/firewall rules on the underlying cloud network) if Pods on different nodes are affected but same-node Pods aren't.

Q10. What's the difference between a Kubernetes NetworkPolicy and a cloud security group?

A security group (or firewall rule) operates at the cloud/VPC level, controlling traffic to/from EC2 instances or nodes based on IPs/ports — it has no concept of Pods, namespaces, or labels. A NetworkPolicy is a Kubernetes-native object enforced by the CNI plugin (not all CNIs support it — Flannel alone doesn't) that controls traffic between Pods based on label selectors and namespaces, which is far more granular and survives Pods being rescheduled with new IPs. In a defense-in-depth setup you typically use both: security groups for the network perimeter, NetworkPolicies for intra-cluster segmentation.

CI/CD Fundamentals

8 questions

Tool-agnostic concepts that interviewers use to gauge whether you understand the principles, not just button-clicking in one specific tool.

Q1. Define Continuous Integration, Continuous Delivery, and Continuous Deployment.

Continuous Integration means developers merge code into a shared branch frequently, with an automated build and test suite running on every change to catch integration issues early. Continuous Delivery extends this so every change that passes CI is automatically built into a deployable artifact and is always in a releasable state — but a human still triggers the actual production release. Continuous Deployment goes one step further and removes that manual gate: every change that passes all automated checks is deployed to production automatically with no human approval step.

Q2. What's a build artifact, and why use an artifact repository instead of rebuilding at each stage?

A build artifact is the immutable output of a build step — a compiled binary, a container image, a packaged library — that should be built exactly once and then promoted unchanged through environments. Rebuilding at each stage risks 'it built differently in prod' bugs from non-deterministic builds, dependency drift, or different builder versions. An artifact repository (Nexus, Artifactory, a container registry, GitLab Package Registry) stores that single build, tagged/versioned, so dev/staging/prod all deploy the byte-for-byte same artifact — this is the core idea behind 'build once, deploy many times.'

Q3. Compare rolling, blue-green, and canary deployment strategies.

Rolling deployment gradually replaces old instances with new ones in place — simple and resource-efficient but a bug affects a growing percentage of traffic as the rollout proceeds, and rollback means rolling forward again rather than an instant switch. Blue-green keeps two full environments (blue=current, green=new); traffic cuts over all at once after the new environment is verified, giving instant rollback by switching back, at the cost of running double infrastructure during the switch. Canary releases the new version to a small percentage of real traffic first, monitors key metrics, and gradually increases that percentage — it catches real-world issues with limited blast radius, but needs good metrics/automation to decide when to proceed or abort.

Q4. How should secrets be handled in a CI/CD pipeline?

Secrets should never be hardcoded in pipeline YAML or committed to the repo. Use the CI platform's built-in protected/masked variable store (GitLab CI/CD variables, Jenkins Credentials, GitHub Actions secrets) scoped to specific branches/environments, or better, fetch short-lived credentials at runtime from a secrets manager (Vault, AWS Secrets Manager) using the pipeline's identity — increasingly via OIDC federation so the pipeline gets a temporary cloud role instead of holding a static long-lived key at all. Logs should be checked for accidental secret leakage, and masking/redaction should be enabled wherever the platform supports it.

Q5. What does 'pipeline as code' mean and why does it matter?

It means the CI/CD pipeline definition itself (stages, jobs, scripts) lives as a versioned file in the repository (.gitlab-ci.yml, Jenkinsfile, GitHub Actions workflow) rather than being configured by hand through a UI. This gives you code review for pipeline changes, history/rollback of pipeline behavior, consistency across branches (each branch can carry its own pipeline definition), and the ability to reuse pipeline logic across repos via templates/shared libraries — the same engineering discipline applied to infrastructure as it is to application code.

Q6. What are some concrete ways to keep a CI/CD pipeline itself secure?

Run jobs with least-privilege service accounts/roles scoped to only what that job needs, rather than one all-powerful credential shared across every job. Pin runner/agent images and dependency versions to avoid supply-chain surprises, and scan both code (SAST) and built artifacts/images (container scanning, dependency/SCA scanning) as pipeline gates rather than after the fact. Protect deployment jobs with branch protections and required approvals for production, sign/verify artifacts where possible (e.g., cosign for container images) so a compromised registry can't silently swap in a malicious image, and treat the CI system itself (its runners, its secret store) as a high-value target worth auditing.

Q7. Why does idempotency matter for deployment scripts/jobs?

An idempotent operation produces the same end state no matter how many times it runs — critical because pipeline retries, re-runs after a transient failure, or parallel triggers are normal operational reality, not edge cases. A deploy script that blindly appends, increments, or assumes a clean starting state will corrupt things on a second run; tools like Terraform (desired-state reconciliation) and `kubectl apply` (declarative, not imperative) are designed to be idempotent by default, which is part of why they're preferred over ad hoc imperative shell scripts for infrastructure changes.

Q8. How do you decide what should be a pipeline gate (blocking) vs a non-blocking check?

Blocking gates should be reserved for checks that protect correctness and security where a failure truly must not reach the next stage — unit/integration test failures, failed security scans above an agreed severity threshold, failed `terraform plan` validation. Non-blocking checks (informational linting, performance benchmarks that fluctuate, experimental scans) should report results visibly (PR comments, dashboards) without halting the pipeline, otherwise teams start tolerating — or worse, bypassing — a pipeline that cries wolf too often. The balance is a frequent point of friction between dev velocity and platform/security teams, and a good answer here shows you've actually had to negotiate it.

GitLab CI/CD Pipelines

8 questions

Expect questions to probe real .gitlab-ci.yml authoring experience — runners, caching, rules, and multi-project pipeline reuse.

Q1. Describe the basic structure of a .gitlab-ci.yml file.

A pipeline is made of stages (e.g., build, test, deploy) that run in order, and jobs assigned to a stage that run in parallel within it unless dependencies say otherwise. Each job defines a `script` (the actual shell commands), an `image` (the container the job runs in, if using the Docker executor), and optionally `rules`/`only`/`except` for conditional execution, `artifacts` to pass files to later stages, and `cache` for speeding up repeated dependency installs.

.gitlab-ci.yml

```
stages:
  - build
  - test
  - deploy

build_job:
  stage: build
  image: node:20
  script:
    - npm ci
    - npm run build
  artifacts:
    paths:
      - dist/

deploy_prod:
  stage: deploy
  script:
    - ./deploy.sh prod
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
      when: manual
```

Q2. What is a GitLab Runner, and what types/executors exist?

A Runner is the agent that actually picks up and executes pipeline jobs — GitLab itself just orchestrates and schedules. Runners can be shared (provided by GitLab.com or a self-managed instance-wide pool), group, or project-specific, and they support different executors: shell (runs directly on the runner's host), docker (spins up a container per job — the most common for isolation/reproducibility), and kubernetes (runs each job as a Pod in a cluster, which scales elastically and is popular for self-hosted setups with variable load).

Q3. Explain `rules` and how it differs from the older `only`/`except`.

`only`/`except` are the legacy way to conditionally include a job based on simple branch/tag/variable matches, but they're inflexible and were deprecated in favor of `rules`. `rules` is a list of conditions evaluated top-to-bottom, each able to set `if` (a CI/CD variable expression), `when` (`on_success`, `manual`, `always`, `never`), and `changes` (only run if specific files changed) — the first matching rule wins, giving much finer-grained, composable control, e.g., running deploy jobs only on `main` and only as a manual approval gate.

rules-example.yml

```
rules:
- if: '$CI_COMMIT_BRANCH == "main"'
  when: manual
- if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
  when: on_success
- when: never
```

Q4. What's the difference between `artifacts` and `cache` in GitLab CI?

`artifacts` are files a job produces that need to be passed forward to later stages or downloaded after the pipeline finishes (build outputs, test reports, compiled binaries) — they're tied to a specific pipeline run and stored by GitLab with a configurable expiry. `cache` is meant purely as a performance optimization to avoid redoing expensive, reproducible work (like `node_modules` or a package manager's download cache) across pipeline runs — it's best-effort and not guaranteed to be present, so a job must still work correctly with a cold cache, whereas artifacts are a hard dependency the next stage can rely on.

Q5. How do GitLab environments and deployment tracking work?

Defining an `environment` block on a deploy job (e.g., `name: production`) tells GitLab to track that job as a deployment to that environment, which gives you a visual deployment history, the ability to see what commit is currently live, one-click rollback to a previous deployment job, and support for environment-scoped CI/CD variables that only apply when deploying to that environment. It's also the mechanism behind GitLab's Review Apps — dynamic, per-merge-request environments spun up automatically and torn down on merge.

Q6. How do you keep secrets out of a GitLab pipeline's YAML and logs?

Store secrets as CI/CD variables in Project/Group Settings, marked Protected (only exposed to pipelines running on protected branches/tags) and Masked (GitLab tries to redact the value from job logs). For anything beyond basic static credentials, integrate with HashiCorp Vault or a cloud secrets manager and use GitLab's OIDC/JWT-based ID token (`id_tokens` in the job) to let the pipeline authenticate and pull short-lived credentials at runtime instead of storing a long-lived secret in GitLab at all — this also limits blast radius if a runner is ever compromised.

Q7. What are ``include`` and ``extends`` used for, and why does it matter for large GitLab setups?

``include`` pulls in CI/CD configuration from other files — local files, other projects, a remote URL, or a CI/CD component — which lets an org maintain a shared, centrally-managed pipeline template (e.g., a standard security-scanning stage) reused across many repositories. ``extends`` lets a job inherit configuration from a hidden job template (prefixed with a dot, like ``.deploy_template``), reducing duplication when several jobs share most of their config but differ in one or two fields — both are key tools for keeping large multi-team pipeline configs DRY and centrally governable.

include-extends.yml

```
.deploy_template:
  image: alpine:3.20
  before_script:
    - apk add --no-cache curl

deploy_staging:
  extends: .deploy_template
  script:
    - ./deploy.sh staging

include:
  - project: 'platform/ci-templates'
    file: '/security-scan.yml'
```

Q8. What is a parent-child (or multi-project) pipeline, and when would you use one?

A parent pipeline can trigger a child pipeline (in the same project, via a generated/dynamic YAML file, or in a different project) using the ``trigger`` keyword, and the child runs as its own independent pipeline with its own stages/jobs. This is useful for monorepos where each service/component should have its own decoupled pipeline that only runs when its part of the repo changes, or for splitting a very large pipeline into manageable, independently-visualized chunks — and ``needs`/`strategy: depend`` lets the parent wait on and reflect the child's pass/fail status.

Jenkins

9 questions

Still extremely common in enterprise environments — expect Jenkinsfile syntax, agent/credential management, and shared library questions.

Q1. Freestyle jobs vs Pipeline jobs — why did the industry move toward Pipelines?

Freestyle jobs are configured entirely through the Jenkins UI (build steps, post-build actions) with no version-controlled definition — the job's logic lives only in Jenkins' internal config, hard to review, diff, or reproduce elsewhere. Pipeline jobs define the same logic as code (a Jenkinsfile) checked into the repository alongside the application, giving code review, history, branch-specific pipeline behavior, and the ability to model complex multi-stage workflows (parallelism, retries, conditional logic) that Freestyle's flat list of build steps can't express well.

Q2. Declarative vs Scripted Pipeline — what's the practical difference?

Declarative Pipeline uses a structured, opinionated syntax (`pipeline { stages { stage('x') { steps {...} } } }`) that's easier to read, validate, and visualize in Blue Ocean, and is the recommended default for most teams. Scripted Pipeline is essentially raw Groovy with Jenkins DSL steps, giving full programmatic control (loops, complex conditionals, custom error handling) at the cost of being harder to lint and reason about. In practice, Declarative is used for the overall structure, with Scripted-style Groovy blocks (`script { ... }`) dropped in for the rare bit of logic Declarative can't express cleanly.

Jenkinsfile

```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps {
        sh 'mvn -B clean package'
      }
    }
    stage('Test') {
      steps {
        sh 'mvn test'
      }
    }
    stage('Deploy') {
      when { branch 'main' }
      steps {
        sh './deploy.sh prod'
      }
    }
  }
}
```

Q3. What is a Jenkinsfile and where does it live?

It's the text file (Groovy-based DSL) that defines a Pipeline job's stages and steps as code, and it should be checked into the same repository as the application it builds/deploys (typically at the repo root), so the build

logic versions and branches alongside the code it builds. The Jenkins job itself is then configured (often via 'Pipeline script from SCM') to just point at the repo and Jenkinsfile path rather than embedding the pipeline logic in Jenkins' own UI/config.

Q4. Explain agents, nodes, and executors in Jenkins.

A node (sometimes called an agent machine) is a physical or virtual machine — or a dynamically provisioned container/pod — that can run builds. An executor is a slot on a node that can run one build/stage at a time; a node with 4 executors can run 4 builds concurrently. The ``agent`` directive in a Jenkinsfile specifies where a pipeline (or an individual stage) should run — ``agent any`` picks any available executor, ``agent { label 'linux' }` targets nodes with a specific label, and ``agent { docker 'image' }` runs the stage inside a fresh container, which is increasingly common for build reproducibility.

Q5. How should credentials be managed in Jenkins?

Credentials should be stored in the built-in Jenkins Credentials store (scoped globally or to a specific folder/project), never hardcoded in a Jenkinsfile or script. The ``withCredentials`` step binds a stored credential to an environment variable only for the duration of that block, and Jenkins automatically masks the value in console logs. For higher-security setups, Jenkins can integrate with HashiCorp Vault or a cloud secrets manager via plugins, so credentials are fetched short-lived at runtime rather than stored long-term inside Jenkins itself.

credentials.groovy

```
withCredentials([usernamePassword(credentialsId: 'docker-hub-creds',
                                usernameVariable: 'DOCKER_USER',
                                passwordVariable: 'DOCKER_PASS')]) {
    sh 'echo $DOCKER_PASS | docker login -u $DOCKER_USER --password-stdin'
}
```

Q6. What are Jenkins Shared Libraries, and why use them?

A Shared Library is a separate Git repository containing reusable Groovy code (custom steps, classes, global variables) that any Jenkinsfile across the org can ``@Library`` and call — used to centralize common logic like a standardized 'build and push image' step, a notification helper, or org-wide security scanning, so individual teams don't reinvent (and inconsistently implement) the same logic. This is Jenkins' answer to the same DRY problem GitLab solves with ``include`` CI/CD components.

Q7. What ways can a Jenkins pipeline be triggered?

Webhooks from the SCM (GitHub/GitLab push or PR events) are the most common and lowest-latency trigger. SCM polling has Jenkins periodically check the repo for changes itself — simpler to set up but slower and wasteful at scale, generally a fallback when webhooks aren't reachable (e.g., Jenkins behind a firewall with no public endpoint). Cron-style scheduled triggers (``triggers { cron('H 2 * * *)' }`) suit nightly builds or scheduled jobs, and pipelines can also be triggered manually or programmatically from another pipeline/job via the ``build`` step.

Q8. How do you run stages in parallel in a Jenkinsfile, and what's a common pitfall?

The `parallel` block inside a stage lets you define multiple branches that run concurrently, each optionally on its own agent — useful for running independent test suites or multi-platform builds simultaneously to cut overall pipeline time. A common pitfall is shared mutable state or a shared workspace being written to by multiple parallel branches at once, causing flaky, hard-to-reproduce failures — each parallel branch should generally get its own workspace/agent or be carefully isolated if they must share one.

parallel.groovy

```
stage('Tests') {
  parallel {
    stage('Unit') {
      steps { sh 'make test-unit' }
    }
    stage('Integration') {
      steps { sh 'make test-integration' }
    }
  }
}
```

Q9. A Jenkins build has been stuck/hanging for an hour. How do you investigate?

Check the build's console output for where it actually stalled — often it's waiting on an interactive prompt that can never be answered in a non-interactive shell, a network call to an unreachable service with no timeout, or a lock/semaphore from a concurrent build on the same resource. Check the agent's executor count and overall load — if all executors on a node are busy, queued builds will appear 'stuck' when they're really just waiting for a slot. As a safeguard going forward, add explicit `timeout()` blocks around risky steps so a hang fails fast and frees the executor instead of blocking the queue indefinitely.

Secrets Management

8 questions

A cross-cutting topic interviewers love because it touches Kubernetes, CI/CD, and IaC all at once — good answers connect the dots between tools.

Q1. Why is storing secrets in plaintext config files or Git fundamentally risky, even in a private repo?

Git history is permanent by default — deleting a secret in a later commit doesn't remove it from history, so anyone with repo access (including past contributors, CI systems, and backups/forks) can retrieve it. Private repos also aren't a strong boundary: access control mistakes, leaked credentials to the Git host itself, or a contractor's offboarding gap can all expose history that was assumed safe. The fix is structural — never let real secret values touch version control in the first place (use placeholders/refs and inject real values at runtime), rather than relying on revoking or rotating after the fact, which only happens once someone notices.

Q2. What is HashiCorp Vault, and what's the difference between static and dynamic secrets?

Vault is a centralized secrets management system that stores, tightly access-controls (via policies tied to an identity/auth method), and audits access to secrets. Static secrets are fixed values you store and retrieve (an API key) — Vault adds access control and audit logging on top, but the value itself doesn't change unless you rotate it. Dynamic secrets are generated on-demand per request with a short TTL — e.g., asking Vault for database credentials gets you a brand-new username/password created just for that lease, auto-revoked when it expires — which drastically shrinks the blast radius if a credential ever leaks, since it's already useless by the time anyone could misuse it.

Q3. Kubernetes Secrets are base64-encoded, not encrypted. What does that mean in practice, and how do you harden them?

Base64 is just an encoding — trivially reversible by anyone with `kubectl get secret -o yaml` access or `etcd` access, it provides no confidentiality on its own. To actually protect Secrets: enable encryption at rest for `etcd` (ideally backed by a cloud KMS so the encryption key itself isn't sitting next to the data), lock down RBAC so only the Pods/users that truly need a Secret can read it (avoid broad `get secrets` permissions at the namespace level), and consider moving the actual secret material outside the cluster entirely using a tool like External Secrets Operator that syncs from Vault/cloud secret managers into Kubernetes Secrets just-in-time.

Q4. Compare Sealed Secrets and External Secrets Operator — when would you pick one over the other?

Sealed Secrets lets you encrypt a Secret client-side with a public key so the encrypted SealedSecret manifest is safe to commit to Git; only the in-cluster controller holding the matching private key can decrypt it back into a real Secret. It's GitOps-friendly and needs no external dependency, but the source of truth is still effectively 'baked into Git' and rotation/sharing across clusters is more manual. External Secrets Operator instead treats an external system (Vault, AWS/GCP/Azure secret managers) as the single source of truth

and continuously syncs values into native Kubernetes Secrets — better for centralized rotation, auditing, and sharing secrets across many clusters/teams, at the cost of depending on that external system being reachable.

Q5. How do cloud-native secret stores (AWS Secrets Manager, Azure Key Vault, GCP Secret Manager) typically fit into an architecture?

They act as the authoritative store for sensitive values, integrated with the cloud's native IAM so access is governed by the same roles/policies used elsewhere in the account — no separate auth system to manage. Workloads retrieve secrets at runtime via SDK calls or a sidecar/CSI driver (e.g., the AWS/GCP/Azure provider for the Secrets Store CSI Driver in Kubernetes) using the workload's own cloud identity (IAM role for a service account, Workload Identity, etc.) rather than a static credential, and most support built-in automatic rotation for things like RDS database passwords.

Q6. How do you securely get a secret into a CI/CD pipeline without long-lived static credentials sitting in the CI platform?

The modern pattern is OIDC federation: the CI platform (GitLab, GitHub Actions, Jenkins with a plugin) issues a short-lived signed JWT identity token for the running job, which is exchanged with the cloud provider (AWS STS AssumeRoleWithWebIdentity, GCP Workload Identity Federation, Azure Federated Credentials) for temporary cloud credentials scoped to a specific role — no static access key stored in CI at all. Where OIDC isn't available, the fallback is the CI platform's masked/protected variable store combined with the secret manager itself, fetching a fresh value per run rather than baking long-lived credentials into the pipeline config.

Q7. What is secret rotation, and what makes it hard to do safely in practice?

Rotation means periodically replacing a credential with a new value and invalidating the old one, limiting how long a leaked credential remains useful. The hard part is usually coordination: every consumer of a secret (multiple services, cached connections, downstream systems) needs to pick up the new value before the old one is revoked, or you get an outage instead of a security win — this is why dynamic secrets (short TTL, generated per-consumer) and tools that support graceful rotation windows (overlap periods where both old and new credentials work) are preferred over a hard cutover for anything stateful like a database password shared by many connections.

Q8. How does least privilege apply specifically to secret access, beyond just 'don't share passwords'?

It means scoping access to secrets as narrowly as the actual need — a CI job should only be able to fetch the specific secret path it needs, for the specific environment it's deploying to, not a blanket read-all-secrets policy; a Pod's service account should only be bound to the RBAC rules and IAM role it actually requires. Concretely this looks like per-path Vault policies, per-namespace RBAC restricting `get`/`list`` on Secrets, and per-environment IAM roles (a staging deploy role can't touch production secrets) — all enforced by the platform, not just by convention or documentation that people are trusted to follow.

Terraform

10 questions

Infrastructure-as-Code fundamentals — state management, the core workflow, module design, and the kinds of production incidents that come from misunderstanding state.

Q1. What is Terraform, and what's the core value proposition of Infrastructure as Code?

Terraform is a declarative IaC tool — you describe the desired infrastructure state in HCL configuration files, and Terraform figures out what API calls are needed to make real infrastructure match that description, across many providers (AWS, Azure, GCP, Kubernetes, and hundreds more) with one consistent workflow. IaC's value is reproducibility (spin up an identical environment from the same code), reviewability (infrastructure changes go through the same PR/code-review process as application code), and an audit trail (every change is a diffable commit, not an undocumented manual console click).

Q2. What is Terraform state, and why does remote state with locking matter for a team?

The state file (terraform.tfstate) is Terraform's record of what real-world resources it's managing and their current attributes — it's how `terraform plan` knows the difference between 'desired config' and 'what actually exists' without re-querying every API for every resource on every run. If state is kept locally, two people running Terraform at the same time (or even one person from two machines) can corrupt it or apply conflicting changes; remote state (S3+DynamoDB, Terraform Cloud, Azure Storage, GCS) with locking ensures only one apply runs at a time and gives the whole team a single source of truth instead of stale, diverging local copies.

backend.tf

```
terraform {
  backend "s3" {
    bucket      = "my-tf-state"
    key         = "prod/network/terraform.tfstate"
    region     = "eu-west-1"
    dynamodb_table = "terraform-locks"
    encrypt     = true
  }
}
```

Q3. Explain the terraform plan / apply / destroy workflow.

`terraform plan` compares your configuration against the current state (and, for managed resources, against real infrastructure) and prints exactly what would be created, changed, or destroyed — without making any real changes, which makes it the primary safety check before applying. `terraform apply` executes that plan (or generates and immediately confirms a new one) against real infrastructure and updates the state file to match. `terraform destroy` tears down everything Terraform manages in that state — used carefully, usually only for ephemeral environments, since it's irreversible for anything without a backup.

Q4. Providers, resources, data sources, and modules — define each.

A provider is the plugin that knows how to talk to a specific API (the AWS provider, the Kubernetes provider) and translates your HCL into real API calls. A resource is something Terraform creates and manages the full

lifecycle of (an EC2 instance, an S3 bucket). A data source is read-only — it looks up information about something that already exists (an AMI ID, an existing VPC) without managing it. A module is a reusable, parameterized collection of resources (and possibly nested modules) that you call multiple times with different inputs — the Terraform equivalent of a function, and the main tool for avoiding copy-pasted infrastructure code across environments.

Q5. How do you avoid putting sensitive values into Terraform state in a recoverable, plaintext way?

State plaintext is somewhat unavoidable by default — Terraform records resource attributes, including ones marked ``sensitive`` (sensitive only suppresses CLI output, it doesn't encrypt the state file itself). The mitigations are: encrypt the remote state backend at rest (S3 SSE, etc.) and restrict who/what can read it via IAM as tightly as you would the secrets themselves; prefer resources that generate secrets outside Terraform's state where possible (e.g., let RDS auto-generate and store a password in Secrets Manager rather than a Terraform-managed ``random_password`` resource living in state); and treat the state backend's access policy as a security-critical asset, not an afterthought.

Q6. What is Terraform drift, and how do you detect and handle it?

Drift happens when real infrastructure diverges from what's recorded in state — e.g., someone manually changed a setting in the cloud console, or an external automated process modified a resource Terraform manages. ``terraform plan`` (especially run on a schedule in CI as a 'drift detection' job) will surface drift as an unexpected diff even when no one touched the Terraform code. Handling it means either updating the configuration to match the manual change if it should stay (then applying so state and config agree), or applying to revert the manual change back to the declared state — but the real fix is process: prevent console access to Terraform-managed resources for anyone except via Terraform.

Q7. What's a provisioner, and why do the Terraform docs themselves recommend avoiding them when possible?

Provisioners (``local-exec``, ``remote-exec``) run arbitrary scripts on a resource at create/destroy time — e.g., SSHing into a new VM to run a setup script. They're a last resort because they're imperative inside a declarative tool: Terraform has no real visibility into what the script does, can't diff or plan its effects, and failures mid-script can leave a resource in a half-configured state that's hard to reconcile on the next plan. Preferred alternatives are baking configuration into the image itself (AMI built with Packer), using cloud-init/user-data for first-boot config, or handing configuration management to a purpose-built tool (Ansible, or a Kubernetes-native approach) rather than Terraform.

Q8. How would you structure a Terraform codebase to support multiple environments (dev/staging/prod) cleanly?

The generally preferred pattern is reusable modules (network, compute, database) parameterized by variables, with a separate top-level directory per environment that calls those modules with environment-specific variable values and its own remote state/backend — this keeps environments fully isolated (a mistake in dev's state can't touch prod's) while sharing the actual infrastructure logic. Terraform Workspaces are an alternative that share one configuration and backend config across multiple named state files, but many teams avoid them for prod-grade separation because it's easy to apply against the wrong workspace by accident — directory-per-environment makes that mistake structurally harder to make.

env-structure

```
envs/  
├── dev/  
│   ├── main.tf      # calls modules/network, modules/app  
│   └── backend.tf    # dev state  
├── staging/  
│   ├── main.tf  
│   └── backend.tf  
└── prod/  
    ├── main.tf  
    └── backend.tf  
modules/  
├── network/  
└── app/
```

Q9. What's the difference between count and for_each, and when would you choose one?

`count` creates N copies of a resource indexed by number (0, 1, 2...) — simple for identical, interchangeable copies, but fragile if you ever remove an item from the middle of the list, since Terraform re-indexes everything after it and can force unnecessary destroy/recreate of unrelated resources. `for_each` creates a resource per item in a map or set, keyed by a stable string rather than a numeric index, so removing one item only affects that specific resource — generally the safer, recommended choice whenever the items aren't truly interchangeable (e.g., creating an IAM user per name in a list).

Q10. How do you handle a failed/partial apply — for example, Terraform crashes or loses connectivity mid-apply?

Terraform applies resources somewhat in dependency order and updates state incrementally as each resource succeeds, so a partial failure usually leaves state accurately reflecting what was actually created, not in a totally broken state — the fix is almost always just to re-run `terraform plan`/`apply`` again, and it will pick up where it left off. If state itself becomes out of sync with reality (rare, but can happen with certain provider bugs or manual intervention), `terraform refresh`` (or `plan -refresh-only`` in newer versions) re-syncs state from real infrastructure, and `terraform state`` subcommands (`rm``, `import``, `mv``) let you manually repair specific entries as a last resort.

Terragrunt

6 questions

A thin DRY/orchestration wrapper around Terraform — questions here check whether you understand what problem it actually solves versus plain Terraform.

Q1. What problem does Terragrunt solve that plain Terraform doesn't handle well?

As a Terraform codebase grows across many environments and modules, you end up repeating the same backend configuration, provider versions, and common input variables in dozens of files — a maintenance burden and a source of drift between environments that should be identical except for a few values. Terragrunt is a thin wrapper that adds DRY configuration inheritance, automatic remote state backend generation, and orchestration across multiple Terraform modules (running plan/apply in dependency order across an entire tree with one command) — all without modifying Terraform itself.

Q2. Describe the typical structure of a terragrunt.hcl file and how it relates to the Terraform module it wraps.

A `terragrunt.hcl` lives alongside (or instead of, depending on style) a Terraform module invocation and specifies a `terraform { source = "..."}` block pointing at the actual module (often a Git URL with a pinned version tag), a `remote_state` block that Terragrunt uses to auto-generate the backend configuration, and an `inputs` block supplying the variables for that specific environment's instance of the module — Terragrunt then generates the Terraform files and runs Terraform commands against them behind the scenes.

terragrunt.hcl

```
terraform {
  source = "git::https://github.com/org/tf-modules.git//vpc?ref=v1.3.0"
}

include "root" {
  path = find_in_parent_folders()
}

inputs = {
  cidr_block = "10.20.0.0/16"
  environment = "staging"
}
```

Q3. How does Terragrunt handle dependencies between modules, e.g., a database module needing a VPC's output?

A ``dependency`` block in one module's `terragrunt.hcl` points at another module's directory and exposes its outputs (e.g., ``dependency.vpc.outputs.vpc_id``) for use as an input — Terragrunt resolves and fetches those outputs automatically rather than you copy-pasting IDs between environments by hand. When running ``run-all``, Terragrunt also uses these dependency blocks to determine the correct execution order across the whole tree, applying the VPC before the database that depends on it, without you manually sequencing separate Terraform runs.

dependency-block.hcl

```
dependency "vpc" {
  config_path = "../vpc"
}

inputs = {
  vpc_id      = dependency.vpc.outputs.vpc_id
  subnet_ids = dependency.vpc.outputs.private_subnet_ids
}
```

Q4. What is ``run-all``, and what's a key safety consideration when using it?

``terragrunt run-all plan`` (or ``apply``/``destroy``) recursively finds every `terragrunt.hcl` under the current directory tree and runs that command across all of them in dependency order, which is hugely convenient for applying an entire environment in one shot instead of ``cd``-ing into N module directories manually. The key risk is exactly that convenience — ``run-all apply`` (or worse, ``run-all destroy``) from too high in the tree can affect far more infrastructure than intended, so most teams scope it carefully, review the plan output closely, and often restrict ``run-all apply``/``destroy`` to CI pipelines with explicit approval gates rather than letting it be run loosely from a laptop.

Q5. How does Terragrunt achieve DRY configuration across many environments?

A root ``terragrunt.hcl`` at the top of the repo defines common configuration — remote state backend settings, provider version constraints generated via a ``generate`` block, common tags — and every environment-specific ``terragrunt.hcl`` deeper in the tree uses ``include { path = find_in_parent_folders() }`` to inherit that shared configuration automatically, only overriding what's actually different for that environment (typically just the ``inputs`` block). This means a backend configuration change happens once at the root instead of being copy-pasted into every environment's file.

Q6. Plain Terraform supports modules and workspaces too — when would a team actually need Terragrunt on top?

Modules and workspaces handle reuse and basic multi-environment state separation reasonably well for small-to-medium setups, but they don't solve cross-module orchestration (running an entire dependency tree of separate state files in the right order) or eliminate repeated backend/provider boilerplate across many module instances — Terragrunt's ``run-all`` and ``include``/``generate`` features specifically target those gaps. Teams managing a handful of environments with straightforward module reuse often don't need it at all; teams managing dozens of environments × many modules, where backend boilerplate and cross-module dependencies become a real maintenance cost, are where Terragrunt earns its place — it's an added tool

and a learning curve, not a free win, so the tradeoff should be deliberate.

OpenTofu

5 questions

A newer area — interviewers ask this to see if you're tracking the IaC ecosystem and understand the licensing context, not just tool syntax.

Q1. What is OpenTofu, and why does it exist as a separate project from Terraform?

OpenTofu is an open-source fork of Terraform, created after HashiCorp changed Terraform's license from the open-source MPL 2.0 to the more restrictive Business Source License (BSL) in 2023, which limited how competing commercial products could use Terraform's code. A group of companies and contributors forked the last MPL-licensed version and formed OpenTofu under the Linux Foundation specifically to keep a genuinely open-source, community-governed IaC tool available, rather than depend on a single vendor's licensing terms.

Q2. Is OpenTofu compatible with existing Terraform configurations and state files?

Yes — at the fork point and for some time after, OpenTofu maintained drop-in compatibility with Terraform HCL syntax, provider plugins (it can use the same providers from the Terraform Registry), and existing state file formats, so most existing Terraform codebases can switch by changing the binary used to run `plan`/`apply`. Over time the two projects have started diverging with their own independent features, so compatibility for any new feature introduced after the fork point should be verified rather than assumed, especially for newer Terraform or OpenTofu-only syntax.

Q3. Name a feature OpenTofu added that wasn't in Terraform at the time, and explain why it mattered.

OpenTofu introduced native state encryption — the ability to encrypt the state file's contents at rest directly within OpenTofu's own workflow, independent of whether the storage backend itself supports encryption. This mattered because, as discussed in the Terraform section, state files routinely contain sensitive attribute values, and relying solely on the backend (S3 bucket encryption, etc.) for protection left a gap for backends or workflows that didn't support it — native encryption closes that gap at the tool level itself.

Q4. Walk through how you'd migrate an existing Terraform codebase to OpenTofu.

In most cases it's a low-risk swap: install the `tofu` CLI, point the same backend configuration at it, and run `tofu init`/`tofu plan` against the existing state to confirm it sees zero unexpected diff before doing anything else. Because the state format was compatible at fork, no state migration step is normally needed for a straightforward switch; the main due-diligence items are checking provider version compatibility, re-running the full test suite/plan across all environments (not just one), and updating CI/CD pipeline definitions and any Terragrunt configuration that hardcodes the `terraform` binary name.

Q5. How would you frame the OpenTofu vs Terraform decision to a team or in an interview answer?

It's mostly a question of governance and licensing risk tolerance versus ecosystem maturity: Terraform has the larger existing user base, more third-party tooling/documentation, and HashiCorp's commercial backing

(Terraform Cloud/Enterprise), while OpenTofu offers a genuinely open license, community governance not controlled by a single vendor, and a few independently-developed features. For an organization already deep into HashiCorp's commercial ecosystem (Terraform Cloud, Sentinel policies), switching has real migration cost for uncertain benefit; for an organization that values license certainty or wants to avoid vendor lock-in, OpenTofu is a credible, low-risk alternative given the compatibility at fork — a good answer shows you can weigh this as a business/risk decision, not just a technical preference.

ArgoCD & GitOps

9 questions

GitOps continuous delivery for Kubernetes — increasingly a default expectation for mid-level platform/DevOps roles, often asked alongside Helm and CI/CD.

Q1. What is GitOps, and how does ArgoCD implement it?

GitOps is a delivery model where Git is the single source of truth for both application config and infrastructure desired state, and an automated controller continuously reconciles the live system to match what's declared in Git — rather than a CI pipeline pushing changes imperatively at deploy time. ArgoCD is a Kubernetes-native GitOps controller: it watches one or more Git repos containing manifests (plain YAML, Helm charts, or Kustomize overlays), continuously compares them against what's actually running in the cluster, and either reports the drift (OutOfSync) or automatically applies the difference, depending on the configured sync policy.

Q2. How does a typical CI/CD flow change when ArgoCD is introduced, compared to a pipeline that runs ``kubectl apply`/`helm upgrade`` directly?

Without GitOps, the CI pipeline itself holds cluster credentials and pushes changes directly — ``kubectl apply`/`helm upgrade`` runs as a pipeline step with a service account that has write access to the cluster. With ArgoCD, CI's job ends at building the artifact and updating a Git repo (often a separate 'config' or 'GitOps' repo) with the new image tag or chart version — ArgoCD then pulls that change into the cluster itself, so the CI system never needs direct cluster credentials at all. This is often summarized as 'push CI, pull CD': CI pushes artifacts, but deployment is pulled by a controller already living inside the cluster, which meaningfully shrinks the pipeline's blast radius if CI credentials are ever compromised.

Q3. Explain ArgoCD's sync policy options: manual vs automated, prune, and self-heal.

By default, ArgoCD only reports drift (OutOfSync) and waits for a human to click Sync — manual sync, good for production environments where you want a deliberate gate before changes land. Automated sync applies new commits to the tracked path as soon as they're detected, with two further toggles: `prune` removes resources from the cluster that were deleted from Git (without it, deleted manifests are silently left running), and `selfHeal` reverts any manual, out-of-band changes made directly to the cluster back to match Git — turning ArgoCD into not just a deployer but an active drift-correction mechanism.

application.yaml

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: web-app
spec:
  source:
    repoURL: https://github.com/org/gitops-config.git
    path: apps/web-app/overlays/prod
    targetRevision: main
  destination:
    server: https://kubernetes.default.svc
    namespace: prod
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

Q4. What is the 'App of Apps' pattern in ArgoCD?

Rather than manually creating one ArgoCD Application resource per microservice, you create a single parent Application whose Git source is itself a directory of child Application manifests — ArgoCD applies the parent, which in turn creates/manages all the child Applications. This gives one place to bootstrap an entire cluster's worth of apps from a single Git commit, makes onboarding a new service as simple as adding one more child Application manifest to the parent's repo, and is a common way to bootstrap a brand-new cluster (ArgoCD installs itself, then the App of Apps installs everything else).

Q5. How would you manage the same application across dev/staging/prod with ArgoCD?

The two common approaches mirror Helm/Terraform's environment patterns: either use Kustomize overlays (a base set of manifests with per-environment patches) or Helm value files per environment, with one ArgoCD Application per environment pointing at the same chart/base but a different overlay path or values file. For many similar environments/clusters, ArgoCD's ApplicationSet controller can generate Applications automatically from a generator (a list, a Git directory structure, or cluster labels/matrix), avoiding hand-maintaining a near-duplicate Application manifest per environment.

Q6. What are sync waves and hooks, and when would you need them?

By default ArgoCD applies all resources in a sync roughly together, which is fine when resources have no ordering dependency, but breaks down when, say, a CRD must exist before a custom resource using it, or a database migration Job must finish before the new app Pods start. Sync waves (an annotation specifying an integer ordering) let you control the sequence resources are applied in within one sync. Sync hooks

(PreSync, Sync, PostSync, SyncFail) let you run one-off Jobs at specific points in the sync lifecycle, very similar in spirit to Helm hooks — and that similarity isn't a coincidence, since Helm charts deployed via ArgoCD can use both Helm's own hooks and ArgoCD's sync waves together if needed.

Q7. How do you handle secrets in a GitOps workflow, given that Git is supposed to hold everything declaratively?

You don't put real secret values in the Git repo ArgoCD syncs from, full stop — the usual pattern combines GitOps with one of the secret-handling tools discussed earlier: either commit Sealed Secrets (encrypted, safe to store in Git, decrypted only by the in-cluster controller) so the GitOps repo stays fully self-contained, or rely on External Secrets Operator to sync real values from Vault/cloud secret managers into native Kubernetes Secrets independently of what ArgoCD syncs — ArgoCD just needs the ExternalSecret **reference** manifest in Git, never the actual secret value.

Q8. An ArgoCD Application shows OutOfSync and won't resolve even after a sync. How do you troubleshoot it?

Check the Application's diff in the UI/CLI (``argocd app diff``) to see exactly which fields differ — often it's a controller or admission webhook mutating the resource after apply (e.g., a defaulted field, an injected sidecar), which ArgoCD then perpetually sees as drift since the live object never matches the Git manifest byte-for-byte. The fix is usually to configure a ``ignoreDifferences`` rule for that specific field/path so ArgoCD stops flagging an expected mutation as drift, rather than fighting the controller that's legitimately changing it. If sync itself is failing outright (not just diffing), check ``argocd app get`` for actual sync errors — RBAC permission issues for the ArgoCD service account, or a genuinely invalid manifest, are common root causes.

Q9. ArgoCD vs Flux — what's the practical difference, at a level a mid-level engineer should know?

Both are CNCF GitOps controllers solving the same core problem (continuous reconciliation of cluster state from Git) and are broadly comparable in capability. ArgoCD ships with a strong built-in Web UI and RBAC/SSO model out of the box, which often makes it the more approachable choice for teams that want visibility for non-CLI users (a visual app topology, click-to-sync). Flux is more UNIX-philosophy/composable — it's a set of smaller controllers (source-controller, kustomize-controller, helm-controller) wired together, more naturally pure-CLI/API driven, and often favored by teams that want tighter integration with their own tooling rather than a standalone UI-centric product. Neither is objectively 'better' — the choice usually comes down to team preference and existing tooling fit.